ENERGIA POWER · PLATFORM ENGINEERING

Home Lab Hub Architecture & Engineering Guide

A reference for software, QA, DevOps and release engineers: how the platform is built, deployed, secured, observed and recovered, and where it goes next.

Document date	November 9, 2026	Repository	`daav817/home-lab-hub`
Hub version	0.1.0.18	Cluster	k3s v1.34.6+k3s1, 3 nodes
Audience	Engineers maintaining or extending the platform	Classification	Internal: contains topology, no secrets

Contents

1 System context and design principles
2 Physical and network topology
3 Cluster platform layer
4 Edge, ingress and request routing
5 Identity and access management
6 The hub application
7 CI/CD and release engineering
8 GitOps and configuration management
9 Data persistence, backup and recovery
10 Observability and alerting
11 Camera and IoT subsystem
12 Security model
13 Operations runbook
14 Known gaps and technical debt
15 Roadmap: remaining phases in detail
16 Platform evolution beyond the phases
A Version inventory
B Namespaces, workloads and ports

Every version, setting and count in this document was read from the running cluster or the repositories on the document date. Secrets, tokens and credentials are omitted. Re-render after significant changes (`python docs/render-pdf.py docs/architecture.html out.pdf`).

1. System context and design principles

Home Lab Hub is a self-hosted application platform: a three-node Kubernetes cluster on commodity mini PCs that runs a family business's line-of-business apps, home automation and video surveillance, behind a single portal (`hub.energiapower.llc`) with centralized identity. The "hub" is both the **portal application** (Next.js) and, by extension, the **platform** it fronts.

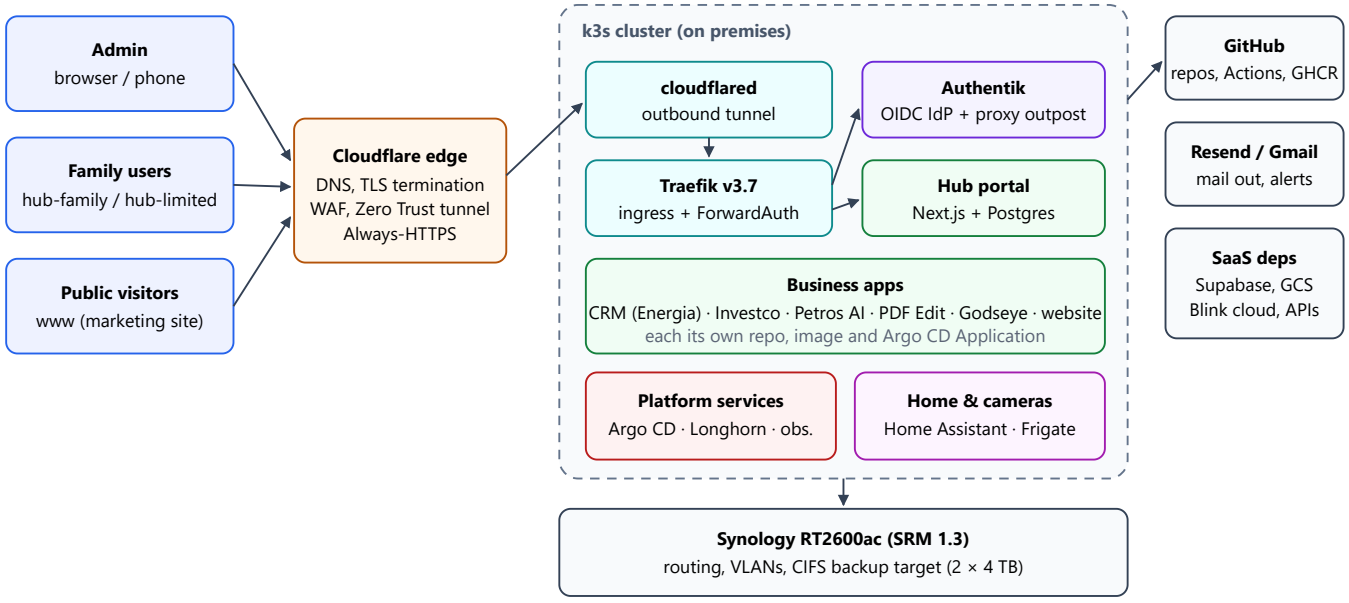


Figure 1. System context. No inbound ports are open on the router: all public traffic arrives over the outbound Cloudflare tunnel.

Design principles (and the decisions they drove)

Principle	Concrete decision
Zero inbound exposure	Cloudflare Tunnel (outbound-only) instead of port forwarding. UPnP removed from the router. Admin-only tools without a public route (Argo CD) are LAN-only.
Git is the source of truth	Every workload is an Argo CD Application with `selfHeal: true` . Secrets are committed as SealedSecrets. Image versions are pinned in git (with one exception, §7).
Centralized identity, minimal double login	Authentik is the IdP. Apps with mature auth keep their own login (no ForwardAuth layered on top), while tools without auth (Prometheus, Alertmanager, Frigate, Longhorn) sit behind Authentik ForwardAuth.
Least privilege for the internet-facing portal	The hub can *read* backup state and *create* snapshots and backups, but cannot delete or restore. Destructive storage operations stay in Longhorn behind a step-up 2FA flow.
Recoverability is tested, not assumed	Nightly Longhorn backups to separate hardware, plus scripted restore tests that compare restored data against live data.
Supply-chain hygiene	Actions pinned by commit SHA; images pinned by tag or digest; third-party HA integrations installed by pinned version and SHA-256; Trivy, Gitleaks and Semgrep in CI.
Commodity hardware, graceful degradation	Three nodes with Longhorn 3-replica volumes for important data; cheaper single-replica class (`longhorn-single`) for bulk, re-creatable data such as camera media.

2. Physical and network topology

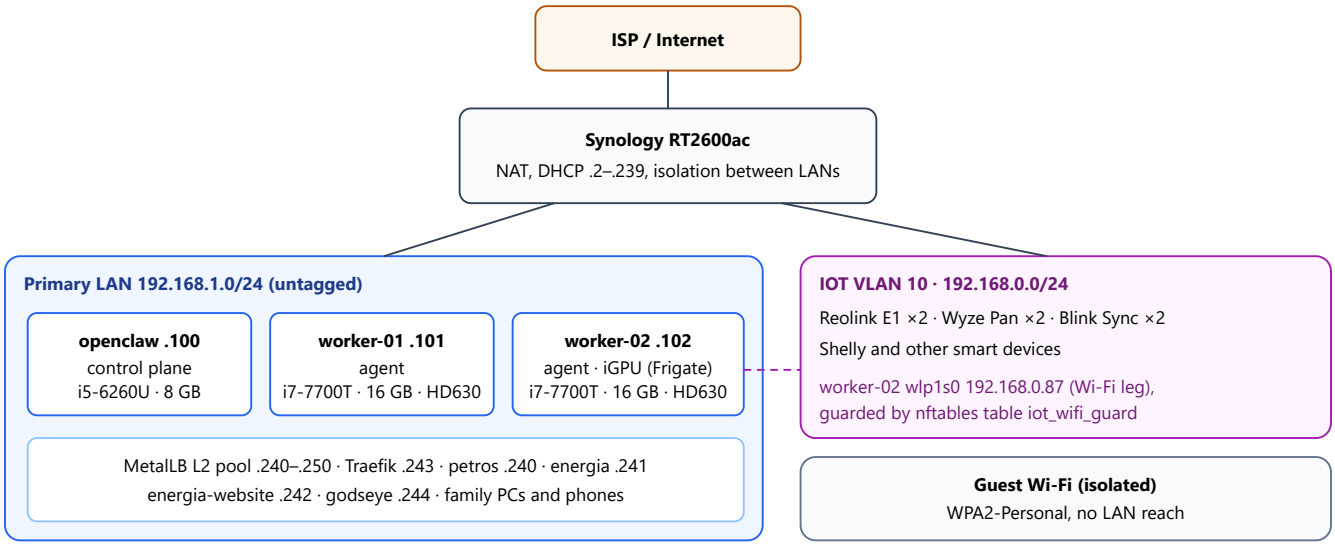


Figure 2. Network topology. The dashed link is worker-02's deliberate Wi-Fi leg into the IOT VLAN, used for Wyze discovery.

Notes engineers must know

- **Router isolation, not firewall rules.** On SRM 1.3, firewall rules don't filter traffic *between* local networks; the per-network *isolation* checkbox does. The six firewall rules created during setup are inert. Isolation is on for all three networks.
- **Calico address autodetection is pinned** (`cidrs: 192.168.1.0/24`). Without this, Calico could pick worker-02's IOT Wi-Fi address as the node IP and break pod networking. This setting lives in the cluster only (documented in the README); see §14.
- **worker-02 nftables guard** (`/etc/iot-wifi-guard.nft` , systemd `iot-wifi-guard`): drops new inbound connections from the IOT VLAN except UDP from the two Wyze cameras, and blocks the camera-bridge ports on `eno1` (TCP 1935, 1984, 5000, 5080, 8554, 8888, 8889, 9997; UDP 8000, 8001, 8189). Changes are validated with `nft -c` against a temporary file before install (`scripts/worker-02-firewall.sh`).
- **Node pinning.** Home Assistant (hostNetwork, needs LAN discovery and a stable address) and Frigate (needs the iGPU and is in the camera firewall allow-list) are pinned to worker-02. Authentik has node affinity that keeps it off the memory-constrained control plane.

3. Cluster platform layer

Component	Version	Managed by	Role and key configuration
k3s	v1.34.6+k3s1	Host install	Single server (openclaw) with two agents. Embedded datastore on the server: the control plane is *not* highly available (§14). Bundled Traefik and ServiceLB are replaced by the components below.
Calico	v3.31.2	Tigera operator	CNI and NetworkPolicy enforcement. Pod CIDR 10.42.0.0/16. Autodetection pinned to the primary LAN CIDR.
MetalLB	v0.14.8	Manifests	Layer-2 LoadBalancer IPs from .240–.250 (outside the router's DHCP range).
Traefik	v3.7.9 (chart 41.1.1)	Helm	Ingress controller on .243. Middlewares for Authentik ForwardAuth. Serves HTTPS internally with its default certificate, which the tunnel trusts with verification disabled.
Longhorn	v1.11.1	Helm	Replicated block storage. Classes: `longhorn` (3 replicas), `longhorn-single` (1), `longhorn-static`, plus `local-path`. Classes are set explicitly on every PVC.
Argo CD	v3.3.8	Manifests	GitOps controller, 13 Applications (§8). UI at `argocd.192.168.1.243.nip.io` (LAN only).
Argo CD Image Updater	v1.1.1	Manifests	Used by Energia and Godseye only: watches GHCR and writes image overrides into the Argo Application.
Sealed Secrets	chart 2.22.0	Argo CD (Helm source)	Controller in `kube-system`. Sealing key backed up offline in an encrypted archive. Never committed.
Intel GPU device plugin	—	Argo CD (cameras)	Exposes `/dev/dri` on worker-02 as `gpu.intel.com/i915` for Frigate's OpenVINO detector.

Resource posture

The workers have headroom (about 46–57% memory used). **openclaw is the constraint**: 8 GB, typically 66–79% used, of which roughly 1.8 GB is the k3s server process and about 3.5 GB sits outside pods. Treat new DaemonSets and any workload without node affinity as a risk to the control plane. Most workloads run a single replica without PodDisruptionBudgets, so a node drain causes short outages (acceptable for a home lab, and noted in §14).

4. Edge, ingress and request routing

All public hostnames are under `energiapower.llc`, proxied by Cloudflare. `cloudflared` (2026.10.3, token-managed, namespace `default`) holds an outbound tunnel; the **hostname → origin routes are configured in the Cloudflare dashboard, not in git** (§14). There are two routing patterns:

Pattern	Tunnel origin	Used by
A. Direct to Service	`http://<svc>.<ns>.svc.cluster.local`	Apps with their own login: hub, CRM, Investco, Petros (`ai.`), PDF Edit, Godseye, website, Grafana, Authentik, Home Assistant.
B. Through Traefik + ForwardAuth	`https://traefik.traefik.svc.cluster.local:443` , "No TLS Verify", Host header preserved	Apps with no login of their own: Prometheus, Alertmanager, Frigate, Longhorn.

> **Rule: never point a pattern-B hostname straight at its Service.**
> Doing so skips the ForwardAuth middleware and publishes an unauthenticated admin tool to the internet. This applies especially to Longhorn, which can delete backups.

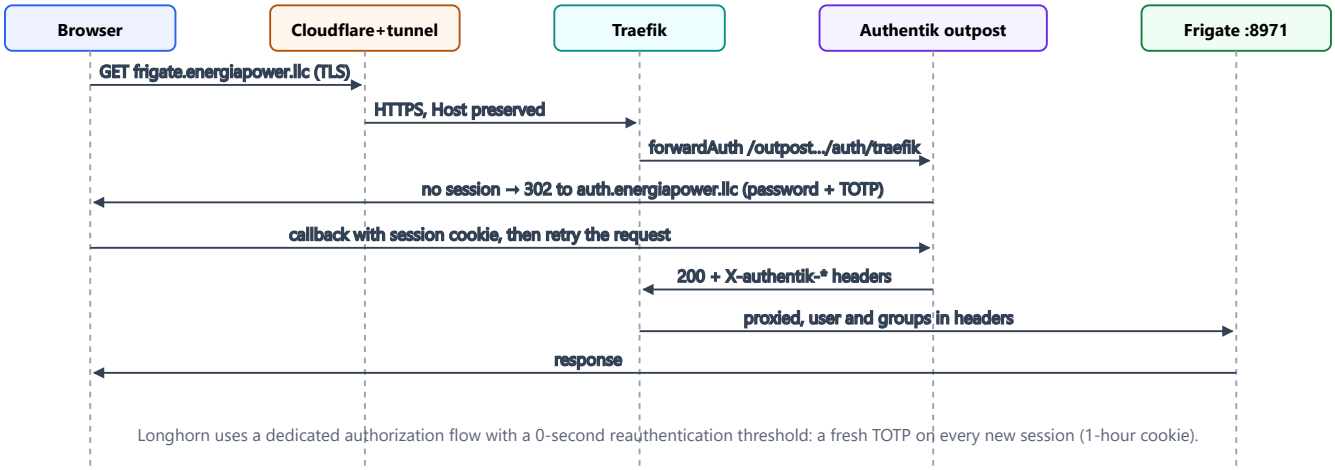


Figure 3. Pattern B: ForwardAuth through Traefik and the embedded Authentik proxy outpost. Frigate trusts the forwarded user headers (header mapping) and runs its own auth only on the internal port.

Operational gotchas

- Renaming a tunnel route can leave its DNS record missing. Check authoritative DNS, and find the active hostnames in cloudflared's "Updated to new configuration" log line.
- Home Assistant sees traffic from pod and node addresses: `trusted_proxies` (10.42.0.0/16 plus the three node IPs) is set in the HA UI, because HA 2026.10 ignores the YAML `http:` block after a one-time import.
- LAN-only access to the hub (`hub.192.168.1.243.nip.io`) goes through a Traefik Ingress. Public access goes through the tunnel (pattern A).

5. Identity and access management

Authentik 2026.8.5

- Helm chart via a multi-source Argo CD Application (`platform/authentik/argocd-application.yaml` , **applied by hand**).
- Own PostgreSQL StatefulSet (NetworkPolicy-restricted). Node affinity keeps it off openclaw.
- Email via Resend as `no-reply@energiapower.llc` . DMARC `p=none` .
- `akadmin` protected with TOTP plus static recovery tokens.

Configuration as code: blueprints

- `platform/authentik/manifests/blueprints.yaml` declares the providers, applications, groups and the outpost's provider list.
- OAuth2 providers on v2026 need `grant_types` set explicitly.
- Validate before pushing with `Importer.validate()` in `ak shell` .

Groups and authorization

Group	Grants	Enforced where
admins	All tiles, the `/manage` pages, and the ForwardAuth apps Prometheus, Alertmanager and Longhorn	Hub (session groups claim); Authentik application policies
hub-family	Every tile with audience `family` , plus Frigate	Hub tile filter (server-side); Frigate provider policy
hub-limited	Tiles with audience `everyone` only (none yet)	Hub tile filter

Hub authentication

The hub is an OIDC relying party using **next-auth 4** (stable line). `src/proxy.ts` (the Next.js 16 replacement for `middleware.ts`) requires a session token on every route except `/api/auth` , `/api/health` , `/sign-in` and static assets. Authorization (`src/lib/access.ts`) maps the groups claim to tile audiences **server-side**: hidden tiles are never sent to the browser. Sign-out is hub-only by design; the Authentik session lasts until the browser closes.

> **Why not ForwardAuth for everything?**
> Apps with their own auth (CRM, Investco, Petros…) would force a double login, and Home Assistant's companion app can't complete a browser-based ForwardAuth flow. The rule is: ForwardAuth only for software that has no authentication of its own.

6. The hub application

Stack

- **Next.js 16.4.5**, React 19.3, TypeScript 5, App Router. Read `node_modules/next/dist/docs` before coding: this major version has breaking changes (see `AGENTS.md`).
- **PostgreSQL 17.11** StatefulSet in `hub`, accessed with the `postgres` (porsager) client.
- Container `node:22-alpine` multi-stage, uid 1001, read-only root filesystem, all capabilities dropped.
- Unit tests: **Vitest 5**. Every `src/lib` module has a test file (access, app-input, apps, backup-history, backups, cluster-health, health, search, status).

Source map

```
src/
  proxy.ts             session gate (all routes)
  instrumentation.ts   startup: run migrations
  app/page.tsx         tile grid, search (Ctrl+K)
  app/manage/          admin CRUD, audit log
    health/            node + workload usage
    backups/           backup status, Back up now
  app/api/health       liveness for probes/tiles
  lib/migrate.ts       ordered SQL migrations
  lib/status.ts        in-cluster tile checks
  lib/kube.ts          Kubernetes API client (SA
token)
  lib/backups.ts       Longhorn CRD reads/creates
```

Data model and migrations

Migrations are an ordered array of SQL strings in `src/lib/migrate.ts`, applied once each at startup inside a transaction guarded by `pg_advisory_xact_lock` (safe with concurrent pods) and recorded in `schema_migrations`. **Never edit an existing migration; append a new one.** Tables: `apps` (name, description, url, status_url, category, audience, hidden, sort_order) and `audit_log` (actor, action, target, detail jsonb). Every change made through Manage writes an audit row.

Status, health and backups

- **Tile status lights** probe each app's `status_url` *from inside the cluster* (Service DNS), not its public URL, so a Cloudflare problem doesn't show as an app outage. The page refreshes every 30 seconds.
- **Health page** reads node and pod metrics through the Kubernetes API using the pod's ServiceAccount token (`lib/kube.ts`), with per-node pod usage (`buildWorkloads`).
- **Backups page** reads Longhorn `volumes`, `backups`, `backuptargets` and `snapshots`, and can *create* snapshots and backups. The RBAC is a namespaced Role in `longhorn-system` (`platform/longhorn-backup/manifests/hub-backup-access.yaml`) with verbs `get`, `list` and `create` only: no delete, no restore.

Extending the hub: checklist

1. Pure logic goes in `src/lib/*.ts` with a matching `*.test.ts`. UI in `src/app`. Server components by default.
2. Schema change: append to `migrations`. Data change: through Manage (audited), not SQL.
3. Needs a new Kubernetes permission: add a narrowly scoped Role in the namespace that owns the resource, never a ClusterRole with wildcards.
4. Run locally: `npm run lint && npm run typecheck && npm test && npm run build`. CI runs the same steps plus `npm audit`, Gitleaks and Semgrep.

7. CI/CD and release engineering

Each application has its own repository, pipeline and Argo CD Application. Release mechanics vary by app today; standardizing them is a roadmap item (§16).

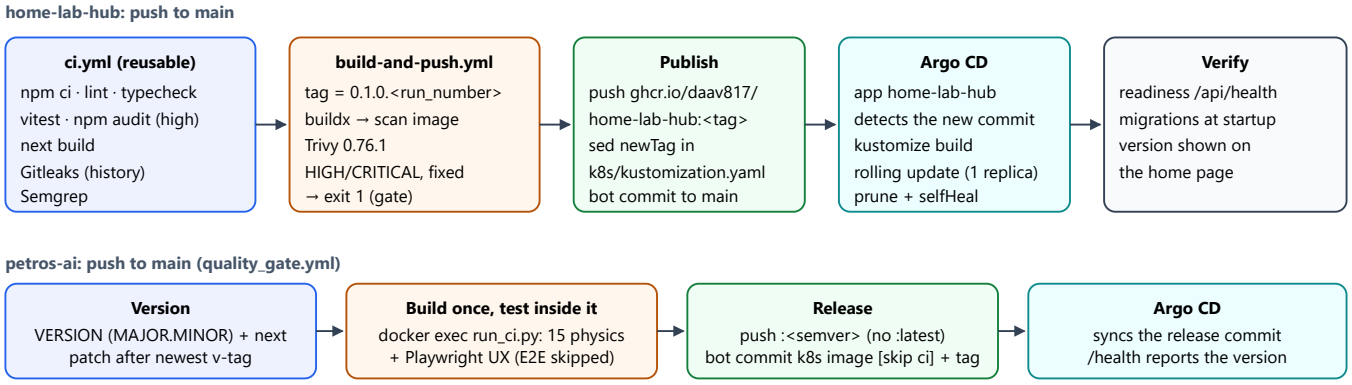


Figure 4. Two reference pipelines. Both end with a bot commit that pins the image version in git; Argo CD deploys from git, never from a floating tag.

Release strategies in use

App	Version scheme	How the cluster learns the version	Quality gate
home-lab-hub	`0.1.0.<run>`	Bot commit to `kustomization.yaml` `newTag`	Lint, types, Vitest, audit, Gitleaks, Semgrep, Trivy
petros-ai	SemVer: `VERSION` + patch from git tags (`v2.4.4`)	Bot commit to `k8s/deployment.yaml` plus annotated tag, `[skip ci]` ; `concurrency` serializes releases	Tests run inside the built image (physics plus Playwright)
investco	`v0.1.x`	`chore(release)` bot commit	Per-repo
energia, godseye	Image tags in GHCR	**Argo CD Image Updater** (`latest` strategy, write-back `argocd`)	Per-repo
pdf-edit, website	Per-repo convention (version.run_number)	Bot commit pinning `newTag`	Per-repo

> **Image Updater with write-back `argocd`**
> The deployed version lives in the Application object, not in git, so git no longer describes what's running for Energia and Godseye. Prefer write-back `git` or the bot-commit pattern (§16).

Release engineering conventions

- **Third-party Actions are pinned by commit SHA**, with the version in a comment. Renovate (hub repo only) proposes bumps. The `protect-main` ruleset blocks deletion and force-push; PR and status-check rules are deliberately off so the bot's pin commit isn't blocked.
- **Bot commits must not trigger a release loop**: commits made with `GITHUB_TOKEN` don't start workflows, and `[skip ci]` covers it anyway. Before committting locally, pull first.
- **Rollback** = revert the bot's pin commit (or set the previous tag) and let Argo CD sync. Never `kubectl set image` : selfHeal reverts it.

- **Petros CI uses the production database** (`DATABASE_URL` secret) for the login test. Test isolation is a roadmap item.

8. GitOps and configuration management

Application	Source	Namespace	Sync policy
home-lab-hub	home-lab-hub · `k8s/` (kustomize)	hub	auto, prune, selfHeal
cameras	home-lab-hub · `platform/cameras/manifests` (kustomize, configMapGenerator)	cameras	auto, prune, selfHeal
home-assistant	home-lab-hub · `platform/home-assistant/manifests`	home-assistant	auto, selfHeal
longhorn-backup	home-lab-hub · `platform/longhorn-backup/manifests`	longhorn-system	auto, selfHeal
monitoring-access	home-lab-hub · `platform/monitoring-access/manifests`	monitoring	auto, selfHeal
authentik	Helm chart 2026.8.5 + home-lab-hub blueprints (multi-source)	authentik	auto, selfHeal
sealed-secrets	Helm chart 2.22.0	kube-system	auto, selfHeal
energia · investco · petros-ai	own repos · `k8s/` (HEAD)	default · investco · default	auto, prune, selfHeal
godseye · pdf-edit · energia-website	own repos · `deploy/k8s` or `k8s/`	godseye · pdf-edit · energia-web	auto, prune, selfHeal

What is *not* in git (configuration drift risk)

Item	Where it lives / how it's changed
Cloudflare tunnel routes and DNS	Cloudflare dashboard (token-managed tunnel)
Monitoring stack, Longhorn, Traefik	Helm releases installed by hand (`helm list -A`)
The Argo Application for Authentik (and other apps' Application objects)	`kubectl apply` by hand (convention: `k8s/argocd-application.yaml` per repo)
Grafana's Loki data source	Grafana's database (added in the UI)
Frigate runtime config	Seeded once from a ConfigMap (`[ -f config.yml ] || cp`). Later changes are written to the PVC after checking that the live file matches the previous seed.
Home Assistant dashboards and integrations	HA `.storage` . The Cameras dashboard is generated by `scripts/make-cameras-dashboard.py` and pasted into the raw editor.
Calico autodetection, worker-02 nftables, k3s flags	Node and cluster state, documented in the README and `scripts/`

Secrets workflow

```
# Values are piped from a hidden prompt; nothing is echoed or written to disk unencrypted.
kubectl create secret generic NAME -n NS --from-literal=KEY="$value" --dry-run=client -o yaml \
  | .tools/kubeseal.exe --format yaml > path/NAME.sealedsecret.yaml   # commit this file
# Helper scripts: scripts/seal-*.py / seal-*.ps1 (MQTT, Neolink, Wyze, camera streams, Petros)
```

Never commit the sealing key, `.tools/` or any backup archive. A SealedSecret is bound to its name and namespace (strict scope), so renaming means resealing.

9. Data persistence, backup and recovery

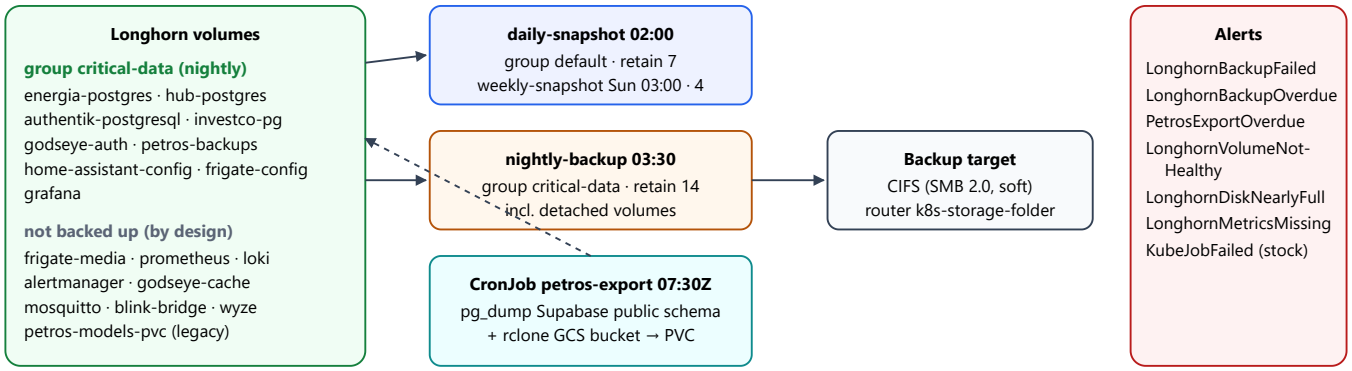


Figure 5. Data protection. Volume group membership is set with PVC labels (`recurring-job-group.longhorn.io/critical-data=enabled`).

Objectives and evidence

Measure	Current state
RPO (data loss window)	≤ 24 h to separate hardware (nightly backup). ≤ 24 h in-cluster (daily snapshot). Petros: ≤ 24 h via the nightly export.
RTO (time to restore)	About 5–15 minutes per database volume (restore into a new PVC from the backup URL, then repoint). Not yet rehearsed as a whole-cluster rebuild.
Restore tests	`scripts/restore-test.ps1` (hub: row counts and MD5 per table match). `scripts/restore-test-energia.ps1` (45 of 45 tables, Nov 9). `scripts/petros-restore-test.yaml` (11 tables; models match active records).
Crash consistency	Longhorn backups are block-level and crash-consistent. Postgres replays WAL on start, which the restore tests exercise. Logical dumps (`pg_dump`) are only used for Petros.
Off-site copy	**None**: the backup target sits in the same building as the cluster. This is the top Phase 7 item.

Restore procedure (reference)

```
1. Longhorn UI (step-up 2FA) or kubectl: find the backup URL for the volume (backups.longhorn.io, label
backup-volume=<pv>).
2. Create a StorageClass with parameters.fromBackup=<url> (see restore-test scripts), then a PVC from it.
3. Scale the app's StatefulSet to 0, swap the claim (or copy data), scale back to 1.
4. Verify with the matching restore-test fingerprint, then remove the temporary StorageClass.
```

10. Observability and alerting

Component	Version	Configuration
kube-prometheus-stack	chart 88.6.3	Prometheus v3.14 (10-day retention; volume removed from the default snapshot group via `scripts/label-metrics-volumes.ps1`), Alertmanager v0.34, Grafana, node-exporter, kube-state-metrics. Helm-managed by hand.
Loki + Promtail	chart 7.3.0 (3.6) / 6.17.1	Logs from all three nodes. **No retention set** (about 0.05 GB/day, 2.6 of 20 GB). Data source added in the Grafana UI.
Alertmanager routing	—	Email through Gmail SMTP to the admin mailbox. Verified with a synthetic test alert.
Exposure	—	Grafana: own login, public. Prometheus and Alertmanager: Authentik ForwardAuth, admins only (`platform/monitoring-access`). Loki: through Grafana only.
Custom rules	—	Longhorn group (§9) in `platform/longhorn-backup/manifests/monitoring.yaml`, plus a Longhorn ServiceMonitor.

Golden signals coverage today: availability (hub tile probes, kube-state-metrics), saturation (node memory and disk, Longhorn disk), and errors (pod restarts, Job failures). Application-level latency and error-rate SLIs are *not* instrumented. Apps don't export metrics, so this is part of the §16 roadmap.

> **Diagnosing a problem: where to look**
> (1) Hub Health page → (2) Grafana dashboards → (3) Loki: `{namespace="cameras", container="bridge"}` → (4) `kubectl describe` / events → (5) Argo CD app diff (drift or a failed sync).

11. Camera and IoT subsystem

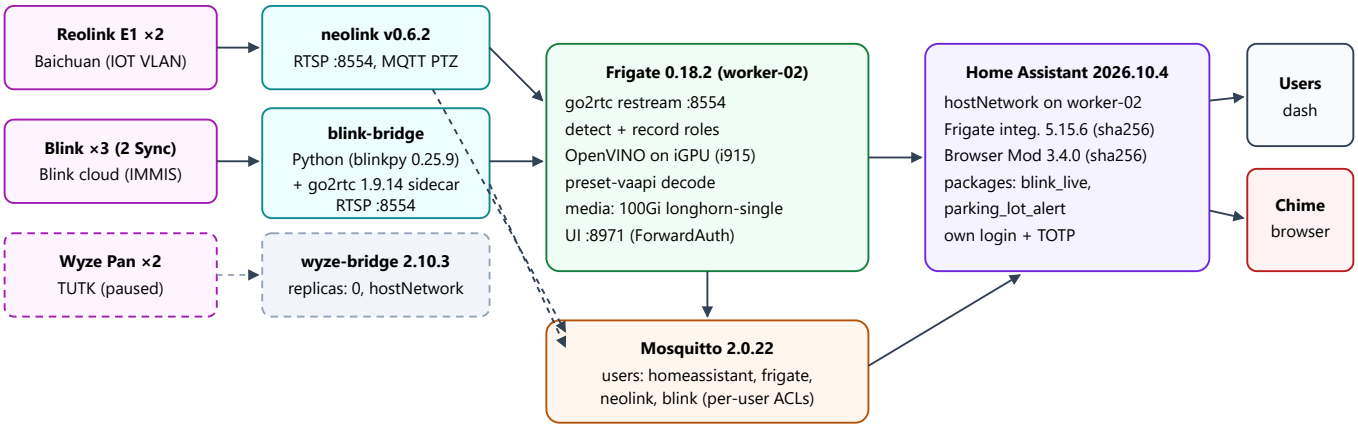


Figure 6. Camera subsystem. Every source normalizes to RTSP, Frigate restreams it, and Home Assistant consumes Frigate's restream.

NetworkPolicies: Frigate UI 8971 from `traefik` and `hub` only; Frigate API 5000 and RTSP 8554 from Home Assistant's node address (192.168.1.102) only; blink-bridge 8554 from the `cameras` namespace and Home Assistant; Mosquitto from the camera namespace and Home Assistant.

The Blink bridge (custom component)

Blink has no local streaming. `platform/cameras/manifests/blink-bridge/bridge.py` holds a single Blink session token (password never stored) and serves each camera as MPEG-TS on a localhost TCP port (18001–18003). The go2rtc sidecar turns that into RTSP using `ffmpeg:tcp://127.0.0.1:1800x#video=copy#audio=aac` ; the audio is re-encoded because Blink's AAC arrives with a 0/0 sample rate. Dependencies are installed with `pip --require-hashes` , and the code ships as a kustomize `configMapGenerator` , so a code change rolls the pod.

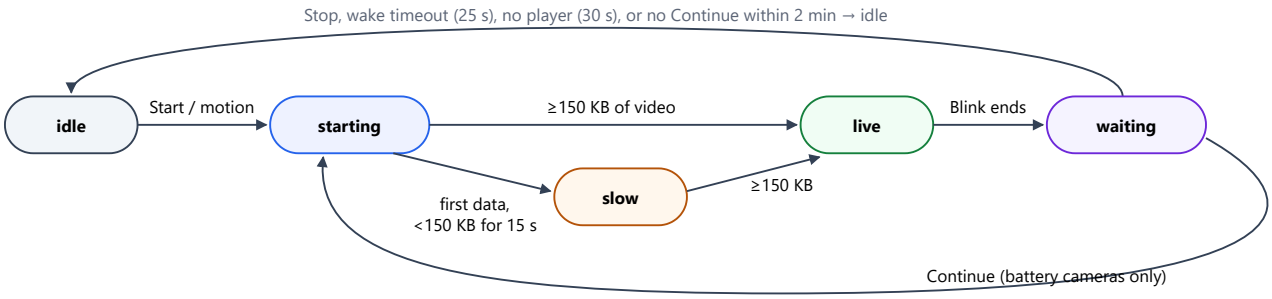


Figure 7. Battery camera session lifecycle. Wired cameras reconnect instead of waiting, with backoff kept on the camera object.

- **Battery protection:** a battery camera's port refuses players unless a session is running, so Frigate's reconnect attempts can never wake a camera. Sessions start only from the MQTT Start button or a Blink motion event.
- **Sync Module arbitration:** Blink allows one live view per Sync Module. A wired camera steps aside while a battery camera on the same `network_id` is in session, and resumes about 2 seconds after it ends.
- **Reconnect tolerance:** a 20-second player grace period, a prewarm buffer for players that connect late, Frigate stream `#timeout=30` and ffmpeg `-timeout 30000000` . The bridge sets go2rtc **preloads** through the local API (wired cameras always; battery cameras while live), so go2rtc's fixed 5-second DESCRIBE deadline is always met.

- **Upstream patch:** blinkpy 0.25.9's `recv` uses `read(n)` and treats split TCP segments as end-of-stream. The bridge replaces it with a `readexactly` loop. **Re-check this when upgrading blinkpy.**

12. Security model

Layer	Controls in place	Residual risk / next step
Edge	Cloudflare proxy and WAF, Always-HTTPS, outbound-only tunnel, no port forwards, UPnP off	Routes managed in a UI (no review trail); in-cluster hops are not end-to-end TLS
Identity	Authentik with mandatory TOTP for admins; invite-only enrollment; step-up 2FA for Longhorn; HA with its own TOTP	Recovery-code storage is manual; no WebAuthn yet; family accounts not yet enrolled
Authorization	Server-side tile filtering; ForwardAuth group policies; hub RBAC limited to read and create on Longhorn	Per-app auth quality varies (Petros auth hardened Nov 6)
Network	Three isolated LANs; worker-02 nftables guard; NetworkPolicies on databases (hub, authentik, investco), cameras and Mosquitto	No default-deny per namespace; several app namespaces have no policy
Workload	Hub: non-root, read-only root filesystem, dropped capabilities, seccomp RuntimeDefault, no SA token automount where unused	Older apps and some vendor images run as root; no Pod Security Admission labels
Secrets	SealedSecrets in git; sealing key offline; values entered through hidden prompts	No rotation schedule; secrets rotated once after a log exposure (Wyze)
Supply chain	SHA-pinned Actions; Trivy, Gitleaks, Semgrep; hash-pinned pip; SHA-256-checked HA add-ons; Renovate	Renovate and scanners on the hub repo only; no image signing (cosign) or SBOMs
Data	Replicated volumes; nightly backups; tested restores; backup-failure alerts	No off-site copy; backups not encrypted at rest on the router

> **Change-control guardrail**
> Automation agents operating on this cluster are subject to a policy classifier. Actions that install external code, or use production credentials from inside pods, need explicit owner approval. Don't build workarounds: ask the owner.

13. Operations runbook

Task	Procedure
Deploy a change	Commit to the repo's `main`. The pipeline builds, tests, pins and pushes, then Argo CD syncs. To check: Argo app is *Synced/Healthy* and the version shows in the app.
Roll back	`git revert` the bot pin commit (or the change itself) and push. For Petros, the previous `v` tag identifies the last good image.
Add a new app to the platform	(1) New repo with Dockerfile and `k8s/` (kustomize). (2) Pipeline that pins a version in git. (3) Copy `ghcr-pull-secret` into the namespace. (4) Apply `k8s/argocd-application.yaml`. (5) Add a Cloudflare route (pattern A or B, §4). (6) Label data PVCs `critical-data`. (7) Add a hub tile with an in-cluster `status_url`. (8) Add a NetworkPolicy for its database.
Add or rotate a secret	Run the matching `scripts/seal-*` helper (hidden prompt), commit the SealedSecret, let Argo sync, restart the consumer if it reads the secret only at start.
Restore a database	§9 procedure; rehearse with `scripts/restore-test*.ps1` first.
Change Frigate config	Edit the seed in `platform/cameras/manifests/frigate.yaml` and commit. Check that the live `/config/config.yml` equals the *previous* seed, then write the new one and restart Frigate.
Change Home Assistant packages	Edit the ConfigMap in `platform/home-assistant/manifests/home-assistant.yaml`. After Argo syncs, restart the HA deployment (the init container copies files on start). Validate with `python -m homeassistant --script check_config -c /config`.
Update the Cameras dashboard	Edit `scripts/make-cameras-dashboard.py`, regenerate, check that the entities exist, paste into the raw configuration editor.
Re-sign Blink	`kubectl exec -it -n cameras deploy/blink-bridge -c bridge -- python /app/bridge.py setup` (email code; stores the token only).
Node maintenance	Cordon and drain workers one at a time. Longhorn rebuilds replicas when the node returns. Draining worker-02 stops Home Assistant and Frigate (pinned).

Known gotchas

- Git Bash on Windows rewrites absolute paths in `kubectl exec` arguments: set `MSYS_NO_PATHCONV=1`. PowerShell 5.1 drops double quotes from native arguments: pipe SQL through stdin.
- kustomize paths must be Windows-style when running locally. Argo CD prunes generated ConfigMaps only on apps with `prune: true`.
- The Wyze bridge log contains auth material: always filter `[AUTH]` and "Auth [" lines when reading it.

14. Known gaps and technical debt

Sev.	Gap	Impact and proposed fix
High	No off-site backup copy	A site-level event (fire, theft) loses the cluster and the backups together. Fix: a second Longhorn backup target (S3-compatible, e.g. Backblaze B2 or Cloudflare R2) for `critical-data`, encrypted, weekly retention (Phase 7).
High	Single control-plane node with 8 GB on openclaw	Losing openclaw stops scheduling and the API; memory pressure is chronic. Fix: move to a 3-server embedded-etcd k3s cluster (the workers become servers too), or upgrade openclaw's RAM and reserve system resources.
Med	Configuration outside git (§8)	Drift and no review trail. Fix: Terraform for Cloudflare; Argo for the Helm releases (monitoring, Longhorn, Traefik); app-of-apps for the Application objects; Grafana provisioning.
Med	Inconsistent release strategies; Image Updater write-back `argocd`	Git doesn't describe what's running for Energia and Godseye. Fix: one reusable release workflow (§16).
Med	Petros CI and the QA Lab use the production database	Tests can mutate production data (the E2E run writes a simulation). Fix: an ephemeral Postgres in CI and a staging Supabase project for the Lab.
Med	No namespace default-deny NetworkPolicies	Lateral movement is possible from a compromised app. Fix: default-deny ingress per app namespace, with explicit allows from Traefik and cloudflared.
Med	Frigate config and HA dashboards are not fully declarative	Manual steps risk drift. Fix: Frigate config rendered on every start (with a migration check); HA dashboards in YAML mode from a ConfigMap.
Low	Loki has no retention; Grafana data source is manual	Unbounded growth (slow). Fix: `retention_period: 30d` and a provisioned data source.
Low	Single replicas, no PDBs	Brief outages during drains. Acceptable at this scale; add PDBs for Authentik and the hub if they gain replicas.
Low	blinkpy monkeypatch; Blink cloud dependency	Upgrades can regress. Fix: upstream the `readexactly` fix; add a bridge smoke test.
Low	Hub tile categories inconsistent (Prometheus and Alertmanager under "Family")	Cosmetic: audience still restricts access. Fix in Manage.

15. Roadmap: remaining phases in detail

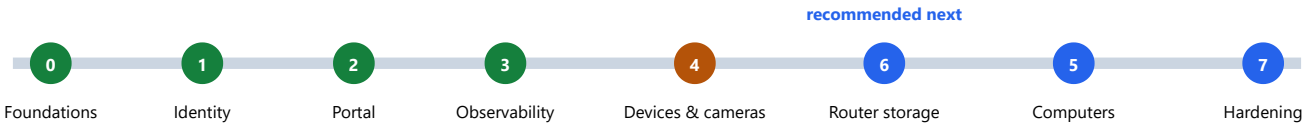


Figure 8. Recommended order: Phase 6 before 5, because it unblocks clip retention (Phase 4) and provides staging for off-site backups (Phase 7).

Phase 4 · Devices & cameras Mostly done

Work item	Design	Acceptance criteria
Wyze Cam Pan	Both cameras answer TUTK discovery, but the session handshake times out from the bridge; the cause looks to be on the camera or Wyze side. Options: (a) docker-wyze-bridge's WebRTC/KVS cloud path instead of local TUTK; (b) Wyze RTSP firmware where available; (c) replace with RTSP-native cameras. The infrastructure (hostNetwork pod, IOT Wi-Fi leg, nftables allow-list) is kept at `replicas: 0`.	Stream stable in Frigate for 24 h; no IOT ports exposed on the primary LAN
Important-clip retention	Use Frigate's export API to write marked clips to an SMB share on the router (needs Phase 6). Triggered from an HA dashboard button (review item → export) or a nightly job for starred events.	A starred clip is on the router within 5 min and survives Frigate retention
Smart devices	Prefer local-push integrations on the IOT VLAN (Shelly via local MQTT or HTTP; Zigbee/Thread via a USB coordinator on worker-02 if needed). Cloud-only devices last. Each device class gets an HA package in git.	Each device controllable from HA with no cloud round-trip where possible
Declarative HA	Move dashboards to YAML mode, generated from `make-cameras-dashboard.py` into the ConfigMap. Keep integrations pinned by SHA-256.	A fresh HA volume rebuilds the dashboards from git
Family rollout	Authentik invitations for `hub-family`; Frigate viewer role mapped from groups.	A family member can view cameras with 2FA and no admin rights

Phase 6 · Router storage Planned, next

Work item	Design	Acceptance criteria
SMB CSI driver	Install `csi-driver-smb` (Argo-managed). StorageClass `router-smb` pointing at dedicated shares (`/clips` , `/shared`), credentials as a SealedSecret, a dedicated SRM user with share-only rights. Keep `k8s-storage-folder` exclusively for Longhorn backups.	A PVC on `router-smb` mounts read-write in a test pod; the backup share stays unreadable to apps
File browser	File Browser (or an equivalent) in its own namespace, behind Authentik ForwardAuth (pattern B), mounting the shared PVC; read-only for `hub-family` , read-write for admins.	Upload and download through hub SSO; no anonymous access

Drives page in the hub	Server-side calls to the SRM web API (read-only account, SealedSecret) for volume usage and SMART status, cached for 5 minutes. New `src/lib/router.ts` plus tests and a `/manage/drives` page. Prometheus alert rules for usage and SMART failures, via a small exporter or the hub publishing metrics.	The page shows both 4 TB drives' usage and health; an alert fires at 85%
Clip export target	Frigate exports and the Phase 4 clip job write to `/clips` with per-year folders and a retention script.	Clips browsable in the file browser

> **Risk**
> The router's CPU and USB-attached drives make a weak primary storage tier. Use SMB for *archival* and *sharing*, never for database volumes, which stay on Longhorn.

Phase 5 · Computers Planned

Work item	Design	Acceptance criteria
Remote access gateway	**Apache Guacamole** (guacd + web + Postgres) with Authentik OIDC, for clientless RDP, VNC and SSH in the browser. Important: **Windows 11 Home cannot host RDP**, so Home-edition PCs need a VNC server or the alternative below. **MeshCentral** (agent-based) works for Home editions and off-LAN machines and adds remote management (power, terminal, files).	Admin opens a PC desktop from the hub with 2FA; sessions are logged
Network path	Egress NetworkPolicy from the gateway namespace to specific PC addresses and ports only (3389, 5900, 22). Wake-on-LAN from an in-cluster helper.	The gateway can reach only the listed hosts
Off-LAN devices	Evaluate a mesh VPN (Tailscale, Headscale or WireGuard) versus the router's VPN Plus (stopped, not uninstalled). Prefer identity-bound access over open VPN ports.	Access without opening inbound ports
Audit	Guacamole session recording to the Phase 6 share; Authentik events to Loki.	Every session is attributable and replayable

Phase 7 · Hardening Planned

Work item	Design	Acceptance criteria
Off-site backups	A second Longhorn backup target (S3-compatible: B2 or R2) with client-side encryption, a separate recurring job (weekly, retain 8) for `critical-data`, and an alert on overdue off-site copies.	A restore test from the off-site target passes
Control-plane resilience	Convert to 3 k3s servers with embedded etcd (snapshots to the router and off-site), or at minimum upgrade openclaw's RAM and set `--system-reserved` / `--kube-reserved`.	Losing any one node keeps the API available (if HA)
Network segmentation	Default-deny ingress in every app namespace; explicit allows from `traefik` and `cloudflared`; egress policies for namespaces holding secrets.	Policy tests (e.g. a netshoot pod) show denied lateral traffic
Workload hardening	Pod Security Admission `restricted` (where vendor images allow) or `baseline`; non-root and read-only filesystems for all first-party apps.	No privileged pods outside system namespaces, except documented ones (HA hostNetwork, GPU plugin)

TLS everywhere	cert-manager with a DNS-01 Cloudflare issuer; valid certificates on Traefik; enable TLS verification on the tunnel's pattern-B routes; optionally Cloudflare Access in front of admin hosts.	No "No TLS Verify" routes remain
Secrets lifecycle	Rotation calendar (Authentik, database passwords, API keys); sealing-key rotation; consider External Secrets with a vault later.	Every secret has an owner and a rotation date
Supply chain	Renovate, Trivy and Gitleaks in every app repo; cosign signing plus Kyverno verification; SBOMs attached to releases.	Unsigned images rejected in app namespaces
Alert review	Runbook links on every alert; a monthly synthetic alert; Alertmanager inhibition rules to avoid alert storms.	Every alert has an owner and a runbook

16. Platform evolution beyond the phases

16.1 Everything in git

- **App-of-apps / ApplicationSet:** a root Application in this repo generates every app's Application (git generator over a list of repos), removing the hand-applied `argocd-application.yaml` step.
- **Helm releases under Argo:** kube-prometheus-stack, Loki, Promtail, Longhorn and Traefik as Argo Helm sources with values files in `platform/`.
- **Cloudflare as code:** Terraform (or the cloudflared config-file mode) for tunnel ingress rules and DNS, reviewed in pull requests, so pattern-B routing can be checked automatically.

16.2 One release workflow for every app

Converge on the Petros model as a **reusable workflow** in a shared `.github` repository: SemVer from `VERSION` plus tags, build once, test inside the image, Trivy gate, push an immutable tag, bot commit the pin plus a git tag, Argo sync. Then retire Image Updater (or switch it to write-back `git`). Add **post-deploy smoke checks** (Argo PostSync hook jobs that hit `/health` and assert the version) and **automatic rollback** on failure.

16.3 Environments and test data

- **Ephemeral CI databases** (service containers) for Petros and others; seed scripts instead of production data.
- A **staging namespace** per app (`<app>-staging`) deployed from a `staging` branch, reachable only on the LAN or through Authentik. Promotion = merge to `main`.
- QA Lab pattern (registry + background runner + live progress) generalized as a **shared test-harness service** for the business apps.

16.4 Observability maturity

- App metrics (OpenTelemetry SDK → Prometheus) for request rate, errors and latency; SLOs for the hub and CRM with burn-rate alerts.
- Distributed tracing (Tempo) once there are service-to-service calls (the broker agent, MLflow).
- A hub "Platform status" page backed by Prometheus queries instead of ad-hoc probes.

16.5 AI and ML workloads

- **MLflow** for Petros: tracking server (Postgres backend, artifacts on GCS or the Phase 6 share), Authentik-protected UI, Petros training runs logging parameters, metrics and model artifacts; the model registry replaces ad-hoc `agent_models` versioning.
- **Energy-broker agent** (RFP → contract): a new app on the standard template, using the CRM's data through an API (not shared database access), human-in-the-loop approval, and audit logging consistent with the hub.
- GPU capacity is limited to the Intel iGPUs (used by Frigate). Training-heavy work should stay on cloud GPUs, with only inference and tracking running here.

16.6 Hub product features

- Ingress / Service auto-discovery (proposed tiles from LoadBalancer Services and tunnel routes), drag-and-drop ordering, per-user favourites.
- Family self-service: invite flow from the hub through the Authentik API (admins only), a device and camera summary widget, a Phase 6 drives widget.

Appendix A · Version inventory (read from the cluster, Nov 9 2026)

Component	Version	Component	Version
k3s / Kubernetes	v1.34.6+k3s1	Argo CD	v3.3.8
Calico	v3.31.2	Argo CD Image Updater	v1.1.1
MetalLB	v0.14.8	Sealed Secrets (chart)	2.22.0
Traefik (chart)	v3.7.9 (41.1.1)	Authentik	2026.8.5
Longhorn	v1.11.1	cloudflared	2026.10.3
kube-prometheus-stack	88.6.3	Prometheus	v3.14.0
Alertmanager	v0.34.0	Loki (chart)	3.6 (7.3.0)
Promtail (chart)	3.5.1 (6.17.1)	Hub: Next.js / React	16.4.5 / 19.3.0
Hub: next-auth	4.24.x	Hub image	0.1.0.18 (node:22-alpine)
Hub / Authentik Postgres	17.11	Energia Postgres	16 (alpine)
Investco Postgres	17 (alpine)	Petros AI	2.4.4
Home Assistant	2026.10.4	Frigate	0.18.2
Frigate HA integration	5.15.6	Browser Mod	3.4.0
Mosquitto	2.0.22	Neolink	v0.6.2 (digest-pinned)
go2rtc (bridge / Frigate)	1.9.14 / 1.9.14	blinkpy	0.25.9 (patched recv)
wyze-bridge	2.10.3 (scaled to 0)	Trivy (CI)	0.76.1

Appendix B · Namespaces, workloads and ports

Namespace	Workloads	Exposure
hub	home-lab-hub (Deployment), hub-postgres (StatefulSet)	Tunnel → Service; LAN Ingress (nip.io)
authentik	server, worker, embedded outpost, authentik-postgresql	Tunnel (auth.); outpost reached by Traefik in-cluster
default	energia, energia-postgres, petros-ai-engine, cloudflared, petros-export (CronJob)	LB .241 / .240; tunnel crm. / ai.
investco	investco, investco-postgres, CronJobs (schwab-sync, levels-snapshot, screener-build)	Tunnel (investco.)
godseye · pdf-edit · energia-web	App deployments (godseye-auth / -cache volumes)	Tunnel; LB .244 / .242
cameras	frigate, neolink, blink-bridge (bridge + go2rtc), wyze-bridge (0), intel-gpu-plugin	Frigate 8971 via Traefik ForwardAuth; RTSP 8554 cluster-internal (policies)
home-assistant	home-assistant (hostNetwork, worker-02), mosquitto	Tunnel (home.); MQTT 1883 internal
monitoring	Prometheus, Alertmanager, Grafana, Loki, Promtail, exporters	Grafana tunnel; Prom / AM via ForwardAuth
longhorn-system	Longhorn manager, engines, UI; recurring jobs	UI via ForwardAuth + step-up flow
argocd	Argo CD, Image Updater	LAN only (Traefik .243, nip.io)
traefik · metallb-system · calico-system · kube-system	Platform controllers, Sealed Secrets	Traefik LB .243

Source of this document: `docs/architecture.html` in the home-lab-hub repository. The plain-language companion guide is `docs/how-it-works.html`.